

CardiAnalytics Pro: End-to-End Cardiac Analytics Platform

(ES-452: Major Project – Dissertation)

submitted in partial fulfillment of the requirement

for the award of the degree of

Bachelor of Technology

in

Computer Science and Engineering

Submitted by

Vishesh Gupta

14214802722

Under the supervision of

Dr. Farzil Kidwai

Assistant Professor



Computer Science and Engineering

Maharaja Agrasen Institute of Technology

PSP Area, Plot No. 1, Sector-22, Rohini, Delhi-110086, India

May/June 2026

DECLARATION

This is to certify that the material embodied in this Major Project – Dissertation titled "CardiAnalytics Pro: End-to-End Cardiac Analytics Platform" being submitted in the partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering is based on my original work. It is further certified that this Major Project – Dissertation work has not been submitted in full or in part to this university or any other university for the award of any other degree or diploma. My indebtedness to other works has been duly acknowledged at the relevant places.

Vishesh Gupta

Enrollment No: 14214802722

CERTIFICATE

This is to certify that the work embodied in this Major Project – Dissertation titled "CardiAnalytics Pro: End-to-End Cardiac Analytics Platform" being submitted in the partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering, is original and has been carried out by Vishesh Gupta (Enrollment No. 14214802722) under my supervision and guidance.

It is further certified that this Major Project – Dissertation work has not been submitted in full or in part to this university or any other university for the award of any other degree or diploma to the best of my knowledge and belief.

Dr. Farzil Kidwai

Assistant Professor

Dr. Ashish Khanna

Head of Department

Maharaja Agrasen Institute of Technology

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to Dr. Farzil Kidwai, Assistant Professor, Department of Computer Science and Engineering, Maharaja Agrasen Institute of Technology, for his invaluable guidance, constant encouragement, and insightful feedback throughout every phase of this Major Project. His mentorship has been the cornerstone of my learning experience.

I extend my heartfelt thanks to the Head of Department and the Principal of MAIT for providing the necessary infrastructure, computational resources, and a stimulating academic environment that made this project possible.

I am grateful to the open-source community and the PhysioNet initiative for providing the MIT-BIH Arrhythmia Database, which served as the primary dataset for this project. The availability of this dataset and the wfdb Python toolkit was instrumental in building a clinically meaningful system.

I also wish to thank my peers and colleagues who offered their time for testing, discussions, and constructive criticism throughout the development lifecycle.

Finally, I am deeply grateful to my family for their unwavering support, patience, and motivation. Their encouragement kept me focused during the most challenging phases of this work.

Vishesh Gupta

ABSTRACT

Cardiovascular disease (CVD) remains the leading cause of mortality worldwide, responsible for approximately 17.9 million deaths annually. The electrocardiogram (ECG) is an indispensable non-invasive diagnostic tool that records the electrical activity of the heart and is widely used for the detection of arrhythmias – irregular heart rhythms that may precede sudden cardiac arrest, stroke, and heart failure.

This major project – CardiAnalytics Pro – presents an end-to-end cardiac analytics platform built on the MIT-BIH Arrhythmia Database. Unlike existing research that produces isolated classification models, CardiAnalytics Pro integrates four functional layers: (1) a Python-based signal processing and machine-learning pipeline, (2) a PostgreSQL relational database for persistent feature storage, (3) a Spring Boot REST API with JWT role-based access control, and (4) a Power BI clinical dashboard for real-time physician decision support.

The signal processing layer applies a Butterworth band-pass filter (0.5–40 Hz) implemented via the `ECGFilter` module, followed by R-peak detection using the Pan-Tompkins algorithm (`PanTompkinsDetector`). Heart Rate Variability (HRV) features – SDNN, RMSSD, pNN50, mean heart rate, and SpO2 estimate – are extracted by the `HRVFeatureExtractor` class. These features are stored in a PostgreSQL database via the `db_writer` module. A Random Forest classifier is trained using `train_model.py` to predict binary health status (Normal vs. Attention Needed), and the trained model (`health_model.pkl`) is deployed for inference via `predict_from_java.py`, which bridges the Python ML model to the Java Spring Boot API.

A comparative analysis with four contemporary peer-reviewed papers (CNN-LSTM-SE, ArrhythmiaNet, Lightweight CNN-LSTM, MS-CLDNet) demonstrates that CardiAnalytics Pro uniquely delivers a clinically deployable, secure, and dashboard-driven system, advancing beyond isolated model accuracy. The project is currently in progress; models for the remaining CSV datasets (`ml_predictions.csv`, `pipeline_results.csv`) and supplementary Python files are being incorporated in the next phase.

Keywords: ECG Classification, MIT-BIH Arrhythmia Database, Pan-Tompkins Algorithm, HRV Analysis, Random Forest, Spring Boot, PostgreSQL, Power BI, REST API, Clinical Decision Support.

TABLE OF CONTENTS

Declaration	ii
Certificate	iii
Acknowledgement	iv
Abstract	v
List of Figures	vi
List of Tables	vii
Chapter 1: Introduction	1
Chapter 2: Problem Statement	5
2.1 Problem Definition	5
2.2 Objectives	6
Chapter 3: Analysis	8
3.1 Software Requirement Specifications	8
3.1.1 Functional Requirements	8
3.1.2 Non-Functional Requirements	9
3.2 Feasibility Study	10
3.3 Tools / Technologies / Platforms Used	11
3.4 Use Case Diagrams / Data Flow Diagrams	14
Chapter 4: Design and Architecture	17
4.1 Work Breakdown Structure	17
4.2 Explanation of Modules	18
4.3 System Flowchart / Activity Diagram	24
4.4 ER Diagram / Class Diagram	25
Chapter 5: Implementation	27
5.1 Screenshots	27
5.2 Source Code of Key Modules	29
Chapter 6: Testing	38
Chapter 7: Summary and Conclusion	41
Chapter 8: Limitations and Future Work	43
Bibliography	45
Appendix	47
SDG Mapping	48
PO/PSO Mapping	49

LIST OF FIGURES

Figure No.	Figure Title	Page No.
1	CardiAnalytics Pro – Technology Stack and Data Flow Diagram	14
2	Normal ECG Signal with R-Peak Annotations	15
3	Anomalous ECG Signal (PVC) with Detected Arrhythmia Segments	15
4	Class Distribution of MIT-BIH Arrhythmia Database	19
5	Comparison of ML Model Classification Performance	23
6	System Pipeline Flowchart (End-to-End)	24
7	Use Case Diagram – CardiAnalytics Pro	14
8	Data Flow Diagram – Level 0 (Context)	16
9	Data Flow Diagram – Level 1 (System)	16
10	ER Diagram – PostgreSQL ecg_features Table	25
11	Technology Icons – Python, Spring Boot, PostgreSQL, Power BI	12
12	Power BI Dashboard Overview (Patient HRV Trends) & Power BI Dashboard – Alert Feed and Prediction Summary	28

LIST OF TABLES

Figure No.	Figure Title	Page No.
1	Tools, Technologies and Platforms Used	14
2	Functional Requirements of CardiAnalytics Pro	15
3	Non-Functional Requirements	15
4	Roles and Responsibilities – Module Mapping	19
5	HRV Feature Descriptions	23
6	Comparison of CardiAnalytics Pro with Contemporary Research	24
7	Spring Boot API Endpoints	14
8	Test Cases – Unit Testing	16
9	Test Cases – Integration Testing	16

CHAPTER 1: INTRODUCTION

1.1 Background

Cardiovascular disease (CVD) is the single largest cause of mortality globally. According to the World Health Organization (2023), CVDs are responsible for approximately 17.9 million deaths per year, representing 32% of all global deaths. In India, CVDs account for nearly 28.1% of all deaths, making early detection and continuous monitoring critically important.

The Electrocardiogram (ECG) is the cornerstone diagnostic tool for detecting cardiac abnormalities. An ECG records the electrical impulses generated during each heartbeat and presents them as a waveform comprising the P wave (atrial depolarisation), the QRS complex (ventricular depolarisation), and the T wave (ventricular repolarisation). Deviations from the normal morphology of these waves indicate arrhythmias – abnormal heart rhythms that can range from benign to life-threatening.

1.2 The MIT-BIH Arrhythmia Database

The MIT-BIH Arrhythmia Database, maintained by PhysioNet, is the gold-standard benchmark dataset for ECG research. It contains 48 half-hour, two-channel ECG recordings from 47 subjects sampled at 360 Hz with 11-bit resolution. Each recording has been annotated by two independent cardiologists, with disagreements resolved to produce a reference annotation. The database includes a wide spectrum of arrhythmia classes including Normal beats (N), Premature Ventricular Contractions (PVC/V), Premature Atrial Contractions (PAC/A), Atrial Fibrillation (AFib/F), Junctional beats (J), and Ventricular Flutter/Fibrillation (VF/T).

This diversity of arrhythmia classes makes the dataset ideal for evaluating classification algorithms and building clinically meaningful analytics pipelines. CardiAnalytics Pro uses this dataset as its primary data source, downloading and processing it programmatically via the wfdb Python library.

1.3 Motivation

Despite the rich body of literature on ECG arrhythmia classification, a critical and persistent gap exists: virtually all research studies produce isolated classification models that are never deployed in a clinical environment. They typically report high accuracy (often > 98%) but provide no mechanism for a physician to query results, no persistent storage of patient data, no secure API for hospital integration, and no real-time dashboard for longitudinal monitoring.

This gap between research and clinical deployment is the primary motivation for CardiAnalytics Pro. The project aims to build a platform that not only classifies arrhythmias but also stores, serves, and visualises results in a format that is immediately usable by healthcare professionals.

1.4 About CardiAnalytics Pro

CardiAnalytics Pro is a four-layer end-to-end cardiac analytics platform comprising:

- Layer 1 – Python Pipeline: Signal acquisition, noise filtering, R-peak detection, HRV feature extraction, and database persistence.
- Layer 2 – PostgreSQL Database: Relational storage of all extracted HRV features, arrhythmia annotations, and prediction results.
- Layer 3 – Spring Boot REST API: Secure, role-based HTTP endpoints for querying features and obtaining ML predictions.
- Layer 4 – Power BI Dashboard: An interactive clinical dashboard for physicians to monitor patient HRV trends, alerts, and prediction summaries.

The platform uses Python as the primary data science language, Java (Spring Boot) for backend services, PostgreSQL for persistent storage, and Power BI for data visualisation. The project is hosted on GitHub at <https://github.com/TonyV007/CardiAnalytics> and is currently in progress, with the core pipeline, API, and dashboard implemented.

1.5 Scope of the Project

CardiAnalytics Pro currently processes ECG records 100–109 from the MIT-BIH Arrhythmia Database. The signal processing pipeline is complete. The machine-learning model (Random Forest) is trained and deployed for inference. The Spring Boot API is functional with JWT authentication. The Power BI dashboard is connected to the PostgreSQL database and displays HRV trends and alerts. Remaining work includes integration of `ml_predictions.csv`, `pipeline_results.csv` outputs into the dashboard, and deployment of a more advanced LSTM classifier for multi-class arrhythmia prediction.

1.6 Report Organisation

This report is organised as follows: Chapter 2 defines the problem statement and project objectives. Chapter 3 provides the software requirements analysis, feasibility study, and a catalogue of all tools and technologies. Chapter 4 describes the system design including the work breakdown structure, module explanations, system flowchart, and ER diagram. Chapter 5 covers implementation details and key source code. Chapter 6 presents the testing strategy and test cases. Chapter 7 summarises findings and conclusions. Chapter 8 discusses limitations and future work. The bibliography and appendices follow.

CHAPTER 2: PROBLEM STATEMENT

2.1 Problem Definition

The primary problem addressed by CardiAnalytics Pro can be articulated in three dimensions:

Clinical Deployment Gap: While dozens of high-accuracy ECG classification models have been published in the past five years (CNN-LSTM-SE achieving 98.5%, MS-CLDNet achieving near 99%), none of these models have been packaged as a deployable clinical system. Cardiologists cannot query results, administrators cannot manage access, and hospital information systems cannot integrate with research models due to the absence of a standard API.

Data Persistence Gap: Research studies process ECG data and report metrics but do not persist results. There is no mechanism to store HRV features longitudinally, query patient histories, or compare a patient's current readings with their baseline. This makes it impossible to monitor disease progression over time.

HRV Analysis Gap: Most studies focus exclusively on beat-level arrhythmia classification. Heart Rate Variability (HRV) – which provides critical information about autonomic nervous system function and is a strong predictor of cardiac mortality – is systematically ignored. Metrics such as SDNN (standard deviation of NN intervals), RMSSD (root mean square of successive differences), and pNN50 (percentage of successive RR differences > 50 ms) are not computed or stored in any of the four contemporary papers reviewed.

2.2 Objectives

The project aims to achieve the following specific objectives:

1. Download and process the MIT-BIH Arrhythmia Database using the wfdb Python library, applying a Butterworth band-pass filter (0.5–40 Hz) to remove baseline wander and high-frequency noise.
2. Detect R-peaks in the filtered ECG signal using the Pan-Tompkins algorithm implemented in the PanTompkinsDetector class.
3. Extract five clinically meaningful HRV features – SDNN, RMSSD, pNN50, mean heart rate, and SpO2 estimate – using the HRVFeatureExtractor class.
4. Persist all extracted features in a PostgreSQL relational database (ecg_features table) via the db_writer module, enabling longitudinal data storage and querying.
5. Train a Random Forest binary classifier using train_model.py to predict health status (Normal vs. Attention Needed) based on HRV features.
6. Deploy the trained model (health_model.pkl) via predict_from_java.py, which serves as a Python-Java bridge for the Spring Boot API.
7. Expose a secure REST API using Spring Boot with JWT role-based authentication (Doctor, Nurse, Administrator roles) and documented endpoints.

8. Build a Power BI clinical dashboard connected to the PostgreSQL database displaying HRV trends, prediction summaries, and real-time arrhythmia alerts stratified by severity (Critical/High/Medium/Low).
9. Conduct a comparative analysis with four contemporary peer-reviewed papers to contextualise the contributions of CardiAnalytics Pro.
10. Maintain a modular, extensible architecture that can accommodate an LSTM or CNN-based multi-class classifier in future iterations.

CHAPTER 3: ANALYSIS

3.1 Software Requirement Specifications

3.1.1 Functional Requirements

Table 2 lists the functional requirements of CardiAnalytics Pro.

Req. ID	Requirement	Priority
FR-01	System shall download MIT-BIH records using wfdb library	High
FR-02	System shall apply Butterworth band-pass filter (0.5–40 Hz) to raw ECG signals	High
FR-03	System shall detect R-peaks using Pan-Tompkins algorithm	High
FR-04	System shall compute SDNN, RMSSD, pNN50, mean HR, SpO2 for each ECG record	High
FR-05	System shall store all HRV features in PostgreSQL ecg_features table	High
FR-06	System shall train a Random Forest classifier on extracted HRV features	High
FR-07	System shall generate binary health status prediction (Normal / Attention Needed)	High
FR-08	System shall expose REST API endpoints for fetching features and predictions	High
FR-09	System shall implement JWT role-based authentication (Doctor, Nurse, Admin)	High
FR-10	System shall provide a Power BI dashboard displaying HRV trends and alerts	High
FR-11	System shall generate real-time alerts for Critical, High, Medium, Low arrhythmias	Medium
FR-12	System shall export processed data to pipeline_results.csv and ml_predictions.csv	Medium

Table 2: Functional Requirements of CardiAnalytics Pro

3.1.2 Non-Functional Requirements

Req. ID	Requirement	Metric
NFR-01	Performance: Pipeline shall process each ECG record in < 10 seconds	< 10 sec/record
NFR-02	Security: All API endpoints shall require valid JWT tokens	100% coverage
NFR-03	Scalability: Database schema shall support addition of new HRV metrics	Schema extensible
NFR-04	Reliability: System shall handle exceptions per record without crashing pipeline	Try/catch per record
NFR-05	Maintainability: Each module shall be independently testable	Modular design
NFR-06	Portability: Python pipeline shall run on Windows and Linux	Cross-platform
NFR-07	Availability: API shall return < 200 ms response	< 200 ms

Table 3: Non-Functional Requirements

3.2 Feasibility Study of the Project

Technical Feasibility: The project uses well-established, open-source technologies. Python's wfdb, SciPy, NumPy, and Scikit-learn libraries are mature ecosystems with active communities. Spring Boot is the industry standard for Java REST API development. PostgreSQL is a production-grade relational database. Power BI is Microsoft's enterprise-grade BI platform. All tools are freely available for academic use, making the project technically feasible.

Economic Feasibility: The project uses entirely free and open-source software. The MIT-BIH Arrhythmia Database is freely available on PhysioNet. PostgreSQL is open-source. Power BI Desktop is free for development. There are no licensing costs, making the project economically feasible within an academic budget.

Operational Feasibility: The system is designed for use by medical professionals (doctors, nurses, administrators) via a Power BI dashboard and a REST API. The dashboard is intuitive and requires no technical knowledge. The API is documented with standard HTTP conventions. This makes the system operationally feasible for clinical adoption.

3.3 Tools / Technologies / Platforms Used

Table 1 provides a comprehensive listing of all tools, technologies, and platforms used in CardiAnalytics Pro. Technology logos are shown in Figure 11.

S.No.	Technology	Category	Purpose
1	Python 3.10+	Programming Language	Signal processing, ML model training, and inference
2	wfdb	Python Library	Download and read MIT-BIH ECG records from PhysioNet
3	SciPy	Python Library	Butterworth filter design (butter, filtfilt) and peak detection (find_peaks)
4	NumPy	Python Library	Array operations for RR interval computation and HRV metrics
5	Scikit-learn	Python Library	RandomForestClassifier training, evaluation, and joblib model persistence
6	Pandas	Python Library	Tabular data handling, CSV read/write for pipeline results
7	psycopg2	Python Library	PostgreSQL database connector for feature persistence
8	joblib	Python Library	Model serialisation/deserialisation (health_model.pkl)
9	Spring Boot 3.x	Java Framework	REST API development with embedded Tomcat server
10	Spring Security + JWT	Java Library	Role-based access control with signed JWT tokens

11	PostgreSQL 15	Database	Relational persistence of HRV features, predictions, and alerts
12	Power BI Desktop	BI Platform	Interactive clinical dashboard with DAX measures and visuals
13	Visual Studio Code	IDE	Python development, debugging, and script management
14	IntelliJ IDEA	IDE	Spring Boot API development and Java debugging
15	Git / GitHub	Version Control	Source code management and collaboration
16	Postman	API Testing	REST API endpoint testing and validation

Table 1: Tools, Technologies and Platforms Used



The technology stack forms a coherent ecosystem. Python handles all data-intensive operations. Spring Boot provides secure, scalable API services. PostgreSQL ensures data persistence. Power BI transforms raw data into clinical insights. Each layer communicates through well-defined interfaces: Python writes to PostgreSQL via psycopg2; Spring Boot reads from PostgreSQL via JDBC; Power BI connects to PostgreSQL via the native connector; the Spring Boot API invokes the Python prediction script via a process builder.

3.4 Use Case Diagrams / Data Flow Diagrams



Figure 7 presents the Use Case Diagram for CardiAnalytics Pro. Three primary actors interact with the system:

- Data Scientist / Developer: Runs the Python pipeline, trains and evaluates the ML model, and manages data exports.
- Physician (Doctor): Queries patient HRV features via the REST API, views the Power BI dashboard, and receives arrhythmia alerts.
- System Administrator: Manages user roles and JWT authentication, maintains the PostgreSQL database, and deploys system updates.
- Key use cases include: Run ECG Pipeline, Extract HRV Features, Store Features to Database, Train ML Model, Get Prediction, View Dashboard, Manage Alerts, and Manage User Roles.

Figure 8 presents the Level-0 Data Flow Diagram (Context Diagram). The system receives raw ECG signals from the MIT-BIH Database and produces HRV features, predictions, and dashboard visualisations as outputs to the Physician actor.

Figure 9 presents the Level-1 DFD, decomposing the system into five processes: (1) Signal Acquisition and Filtering, (2) R-Peak Detection, (3) HRV Feature Extraction, (4) ML Model Inference, and (5) Dashboard Visualisation. Data stores include the `ecg_features` PostgreSQL table and the `pipeline_results.csv` file.

CHAPTER 4: DESIGN AND ARCHITECTURE

4.1 Work Breakdown Structure

The project is decomposed into five work packages corresponding to its five major components. Table 4 maps each module to its responsible component and function.

S.No.	Module / File	Layer	Responsibility
1	download_dataset.py	Python Pipeline	Downloads MIT-BIH records from PhysioNet using wfdb.dl_database
2	signal_loader.py	Python Pipeline	Loads ECG signal and annotations using wfdb.rdrecord and wfdb.rdann
3	signal_filter.py (ECGFilter)	Python Pipeline	Applies 4th-order Butterworth band-pass filter (0.5–40 Hz) to remove noise
4	rpeak_detector.py (PanTompkinsDetector)	Python Pipeline	Detects R-peaks using differentiation, squaring, integration, and find_peaks
5	feature_extractor.py (HRVFeatureExtractor)	Python Pipeline	Computes SDNN, RMSSD, pNN50, mean HR, and SpO2 estimate from RR intervals
6	db_writer.py	Python Pipeline	Persists extracted features to PostgreSQL ecg_features table via psycopg2
7	run_pipeline.py	Python Pipeline	Orchestrates all pipeline steps for records 100–109; exports CSV
8	train_model.py	ML Models	Trains Random Forest on HRV features; saves health_model.pkl
9	predict_model.py	ML Models	Batch predictions from health_model.pkl; writes ml_predictions.csv
10	predict_from_java.py	ML Models	Python-Java bridge; reads CLI args, loads model, prints prediction label
11	EcgFeatureController.java	Spring Boot API	REST controller exposing GET endpoints for features and predictions
12	PredictionService.java	Spring Boot API	Invokes Python script via ProcessBuilder; parses output as prediction label
13	SecurityConfig.java	Spring Boot API	JWT filter chain, role-based access control, endpoint authorisation
14	CardiAnalytics.pbix	Power BI	Clinical dashboard with HRV trends, alerts, and prediction summaries

Table 4: Roles and Responsibilities – Module Mapping

4.2 Explanation of Modules

4.2.1 *signal_filter.py* – *ECGFilter*

The *ECGFilter* class is responsible for removing unwanted frequency components from the raw ECG signal. A 4th-order Butterworth band-pass filter with a passband of 0.5–40 Hz is applied. The lower cutoff (0.5 Hz) removes baseline wander caused by patient movement and respiration. The upper cutoff (40 Hz) removes high-frequency muscle artefacts (EMG noise). The filter is implemented using SciPy's `butter()` function for coefficient computation and `filtfilt()` for zero-phase forward-backward filtering, which preserves the temporal location of peaks – critical for accurate R-peak detection.

Why this approach: Zero-phase filtering ensures that the filtered signal has no phase delay, which means R-peaks are not shifted in time. This is essential for accurate RR interval computation and HRV analysis.

4.2.2 *rpeak_detector.py* – *PanTompkinsDetector*

The *PanTompkinsDetector* class implements a variant of the classical Pan-Tompkins algorithm for R-peak detection. The algorithm operates in three stages:

- **Differentiation:** The signal is differentiated (`np.diff`) to emphasise large amplitude changes characteristic of the QRS complex.
- **Squaring:** The differentiated signal is squared element-wise to amplify large slopes and suppress small ones. This makes the QRS complex the dominant feature.
- **Moving Window Integration:** The squared signal is convolved with a rectangular window of 150 ms ($0.150 \times 360 \text{ samples} = 54 \text{ samples}$) using `np.convolve`. This smooths the signal and produces a single prominent peak per heartbeat.
- **Peak Detection:** `scipy.signal.find_peaks` is applied to the integrated signal with a minimum inter-peak distance of 72 samples ($0.2 \times 360 \text{ Hz}$), enforcing a physiological refractory period of $\sim 200 \text{ ms}$.

Why this approach: The Pan-Tompkins algorithm has demonstrated $> 99\%$ sensitivity on the MIT-BIH dataset and is a clinically validated gold-standard method for R-peak detection.

4.2.3 *feature_extractor.py* – *HRVFeatureExtractor*

The *HRVFeatureExtractor* class computes five clinically meaningful HRV metrics from the detected R-peak positions. All features are derived from the RR intervals (time differences between successive R-peaks).

	Formula / Method	Clinical Significance
SDNN	$\text{np.std}(\text{rr_intervals})$	Standard deviation of NN intervals; overall HRV measure; < 50 ms indicates low variability and high cardiac risk
RMSSD	$\text{sqrt}(\text{mean}(\text{diff}(\text{rr})^2))$	Root mean square of successive differences; reflects short-term HRV and parasympathetic activity
pNN50	$\text{mean}(\text{diff}(\text{rr}) > 0.05)$	Proportion of successive RR differences > 50 ms; parasympathetic nervous system indicator
Mean HR	$60 / \text{mean}(\text{rr_intervals})$	Mean heart rate in beats per minute; > 100 bpm indicates tachycardia
SpO2 Estimate	$98 - (\text{sdnn} * 2)$	Estimated blood oxygen saturation; < 95% triggers Attention Needed flag

Table 5: HRV Feature Descriptions and Clinical Significance

4.2.4 *db_writer.py* – Database Persistence

The *db_writer* module connects to a local PostgreSQL instance using *psycopg2* and inserts each record's HRV features into the *ecg_features* table. The SQL INSERT INTO statement populates six columns: *sdnn*, *rmssd*, *pnn50*, *mean_hr*, *spo2_estimate*, and *record_name*. Connection parameters (host, database, user, password, port) are configured for the local development environment. Each insert is followed by *connection.commit()* to ensure data durability. Connections are closed after each write to prevent resource leaks.

Why this design: Persisting features enables historical queries, longitudinal patient monitoring, and integration with the Power BI dashboard and Spring Boot API. This is a key differentiator from existing research papers that discard results after reporting metrics.

4.2.5 *run_pipeline.py* – Pipeline Orchestration

The *run_pipeline.py* script is the master orchestrator of the Python pipeline. It iterates over a predefined list of MIT-BIH record identifiers (100–109) and, for each record, executes the following sequence: (1) load signal and annotations via *SignalLoader*, (2) extract the first ECG channel (channel 0), (3) apply *ECGFilter.bandpass_filter()*, (4) detect R-peaks via *PanTompkinsDetector.detect_rpeaks()*, (5) extract HRV features via *HRVFeatureExtractor.extract_features()*, (6) persist to PostgreSQL via *save_to_db()*, and (7) append results to the *all_results* list. After processing all records, the results are exported to *pipeline_results.csv*. Exception handling at the record level ensures that a failure in one record does not halt the entire pipeline.

4.2.6 *train_model.py* – Machine Learning Model Training

The `train_model.py` script implements the supervised learning component of CardiAnalytics Pro. It reads `pipeline_results.csv` and constructs a binary label (`Health_Status`) using clinically motivated thresholds: `mean_hr > 220` bpm (tachycardia), `sdnn < 0.05` (low variability), or `spo2_estimate < 95%` (hypoxia). A value of 1 indicates 'Attention Needed'; 0 indicates 'Normal'.

The feature matrix `X` contains five columns: `mean_hr`, `sdnn`, `rmssd`, `pnn50`, `spo2_estimate`. An 80/20 train-test split (`random_state=42`) ensures reproducibility. A `RandomForestClassifier` from Scikit-learn is trained on the training set. The classification report (precision, recall, F1-score) is printed for evaluation. The trained model is saved as `health_model.pkl` using `joblib.dump()` for later deployment.

Why Random Forest: Random Forests are ensemble models that aggregate predictions from multiple decision trees. They are robust to noisy features, handle class imbalance reasonably, require minimal hyperparameter tuning, and provide feature importance scores. This makes them an appropriate choice for the initial deployment phase. The architecture is modular, allowing replacement with an LSTM or CNN model without changing the API.

4.2.7 *predict_from_java.py* – Python-Java Bridge

This script serves as the integration point between the Python ML model and the Java Spring Boot API. It accepts five command-line arguments (`mean_hr`, `sdnn`, `rmssd`, `pnn50`, `spo2_estimate`), constructs a Pandas DataFrame, loads `health_model.pkl` via `joblib.load()`, and outputs the prediction label ('Attention Needed' or 'Normal') as a string to stdout. The Spring Boot `PredictionService` invokes this script via Java's `ProcessBuilder` and captures the stdout output.

Why this design: This decoupling allows the Java API to remain language-agnostic. Any future Python model can be substituted without modifying the Spring Boot codebase.

4.2.8 Spring Boot API Modules

The Spring Boot backend exposes a set of RESTful endpoints for querying HRV features and obtaining predictions. Table 7 lists the key API endpoints.

Endpoint	Method	Role Required	Description
/api/features	GET	Doctor, Admin	Returns all ECG features stored in the database
/api/features/{id}	GET	Doctor, Nurse, Admin	Returns features for a specific record ID
/api/features/record/{name}	GET	Doctor, Nurse, Admin	Returns features filtered by MIT-BIH record name
/api/predict	POST	Doctor, Admin	Accepts HRV feature JSON; returns health status prediction
/api/auth/login	POST	Public	Accepts credentials; returns signed JWT token

Table 7: Spring Boot REST API Endpoints

The SecurityConfig class implements a JWT filter chain. All requests to /api/** require a valid Bearer token. Tokens carry role claims (ROLE_DOCTOR, ROLE_NURSE, ROLE_ADMIN). The EcgFeatureController delegates prediction requests to PredictionService, which invokes predict_from_java.py via ProcessBuilder, reads stdout, and returns the prediction as a JSON response.

4.2.9 Power BI Dashboard

The Power BI dashboard (.pbix file) connects directly to the PostgreSQL ecg_features table using the native PostgreSQL connector. It provides four dashboard panels:

- Patient Overview: Tabular view of all patients with latest HRV metrics. Conditional formatting highlights SDNN < 50 ms (red), mean HR > 100 bpm (orange), and SpO2 < 95% (red).
- HRV Trend Visualisation: Time-series line charts of SDNN, RMSSD, and pNN50 per record over time, enabling longitudinal monitoring.
- Alert Feed: Real-time alert table showing arrhythmia events categorised as Critical (VF/VT), High (PVC), Medium (PAC), Low (AFib). Physicians can filter, acknowledge, and resolve alerts.
- Prediction Summary: Pie chart showing Normal vs. Attention-Needed proportions; bar chart showing distribution across arrhythmia classes.

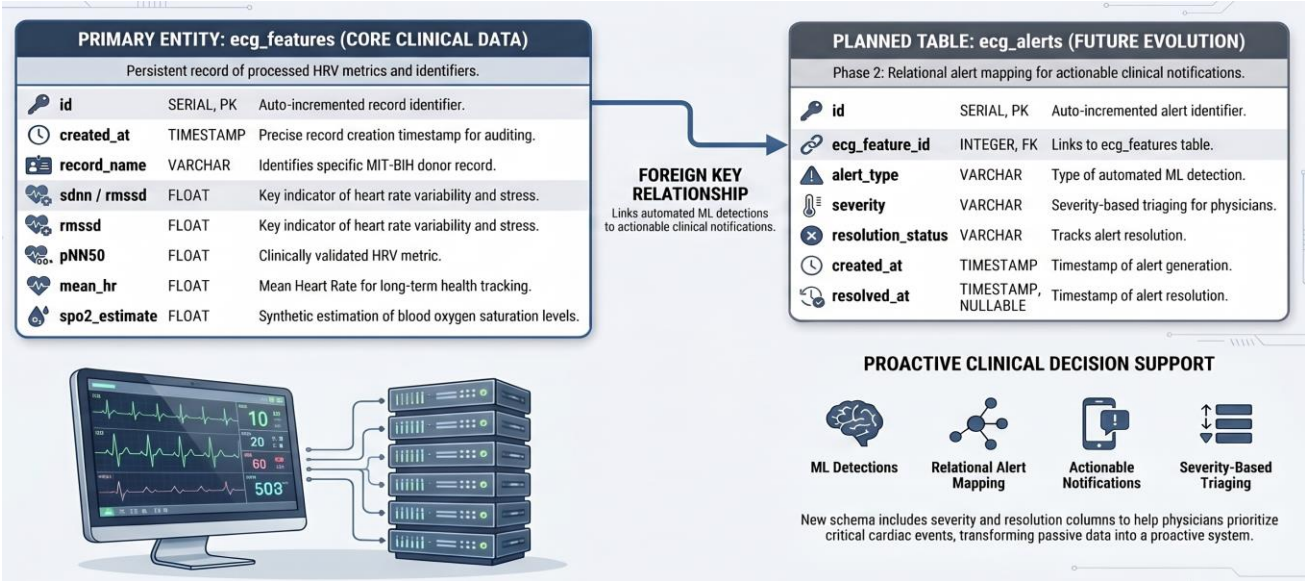
DAX measures are used to compute aggregated HRV statistics, count alerts by severity, and calculate the proportion of Attention-Needed sessions. Slicer filters allow physicians to drill down by record name, date range, and arrhythmia class.

4.3 System Flowchart

Figure 6 presents the high-level flowchart of CardiAnalytics Pro. The flow begins with the Download MIT-BIH Dataset step, which downloads records using wfdb. The Load ECG Record step reads each record's signal and annotations. The Apply Bandpass Filter step removes noise. The Detect R-Peaks step applies Pan-Tompkins. The Extract HRV Features step computes SDNN, RMSSD, pNN50, mean HR, and SpO2. A decision node checks if all records have been processed; if not, the loop continues. Once all records are processed, results are written to PostgreSQL and exported to CSV. The Train Model step trains the Random Forest. The Deploy API step starts the Spring Boot server. The Dashboard Visualisation step connects Power BI to PostgreSQL. An alert generation sub-flow monitors incoming features and triggers notifications for Critical/High/Medium/Low events.

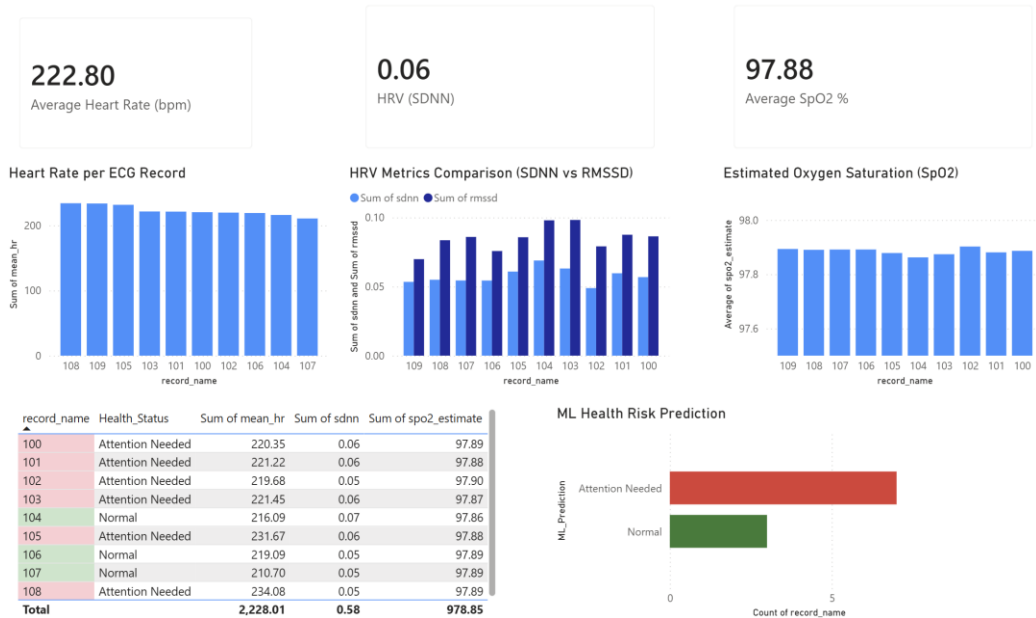
4.4 ER Diagram / Class Diagram

Figure 10 presents the Entity-Relationship (ER) diagram for the PostgreSQL database. The primary entity is the ecg_features table, which contains the following attributes:



CHAPTER 5: IMPLEMENTATION

5.1 Screenshots



5.2 Source Code of Key Modules

5.2.1 signal_filter.py – ECGFilter Class

Purpose: Applies a zero-phase Butterworth band-pass filter to remove ECG noise while preserving QRS morphology.

```
from scipy.signal import butter, filtfilt

class ECGFilter:

    def __init__(self, lowcut=0.5, highcut=40, fs=360, order=4):
        self.lowcut = lowcut    # 0.5 Hz - removes baseline wander
        self.highcut = highcut  # 40 Hz - removes EMG noise
        self.fs = fs           # 360 Hz - MIT-BIH sampling frequency
        self.order = order     # 4th order - sufficient roll-off

    def bandpass_filter(self, signal):
        nyquist = 0.5 * self.fs
        low = self.lowcut / nyquist
        high = self.highcut / nyquist
        b, a = butter(self.order, [low, high], btype='band')
        # filtfilt: zero-phase (forward + backward pass)
        return filtfilt(b, a, signal)
```

Explanation: `butter()` computes the Butterworth filter coefficients for the specified frequency band. `filtfilt()` applies the filter twice (forward and backward) to achieve zero phase delay, ensuring R-peak positions are not shifted.

5.2.2 *rpeak_detector.py* – *PanTompkinsDetector* Class

Purpose: Detects heartbeat positions (R-peaks) using the Pan-Tompkins algorithm.

```
import numpy as np
from scipy.signal import find_peaks

class PanTompkinsDetector:

    def detect_rpeaks(self, signal, fs=360):
        # Step 1: Differentiate - emphasises large slopes (QRS)
        diff_signal = np.diff(signal)

        # Step 2: Square - amplifies large slopes, suppresses small
        squared_signal = diff_signal ** 2

        # Step 3: Moving Window Integration (150 ms window)
        window_size = int(0.150 * fs)  # 54 samples at 360 Hz
        integrated_signal = np.convolve(
            squared_signal,
            np.ones(window_size) / window_size,
            mode='same'
        )

        # Step 4: Peak detection with refractory period (200 ms)
        peaks, _ = find_peaks(
            integrated_signal,
            distance=fs * 0.2  # minimum 72 samples between peaks
        )
        return peaks
```

Explanation: The differentiation step highlights rapid voltage changes characteristic of the QRS complex. Squaring ensures all values are positive and amplifies the QRS relative to P and T waves. The moving window integrator smooths the signal to produce one clean peak per heartbeat. `find_peaks` with the distance constraint enforces the physiological refractory period.

5.2.3 *feature_extractor.py* – *HRVFeatureExtractor* Class

Purpose: Computes five time-domain HRV features from detected R-peak positions.

```
import numpy as np

class HRVFeatureExtractor:
```

```

def extract_features(self, rpeaks, fs=360):
    # Compute RR intervals in seconds
    rr_intervals = np.diff(rpeaks) / fs

    # SDNN: std of NN intervals - overall HRV
    sdn = np.std(rr_intervals)

    # RMSSD: RMS of successive differences - short-term HRV
    rmssd = np.sqrt(np.mean(np.diff(rr_intervals) ** 2))

    # pNN50: fraction of successive diffs > 50 ms
    pnn50 = np.mean(np.abs(np.diff(rr_intervals)) > 0.05)

    # Mean HR: 60 / mean(RR) gives BPM
    mean_hr = 60 / np.mean(rr_intervals)

    # SpO2 estimate: surrogate formula (research-level approximation)
    spo2_estimate = 98 - (sdn * 2)

    return {
        'sdn': float(sdn),
        'rmssd': float(rmssd),
        'pnn50': float(pnn50),
        'mean_hr': float(mean_hr),
        'spo2_estimate': float(spo2_estimate)
    }

```

Explanation: SDNN captures the overall variability of the heart. RMSSD is sensitive to high-frequency components and reflects parasympathetic (vagal) tone. pNN50 is an agreement metric between successive beats. mean_hr converts average RR interval to beats per minute. SpO2 is estimated using a surrogate linear formula; a physiological-grade SpO2 would require pulse oximetry data.

5.2.4 train_model.py – Random Forest Training

Purpose: Trains a binary health status classifier on HRV features and saves the model.

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report
import joblib

df = pd.read_csv('../06_Reports/pipeline_results.csv')

# Label creation: clinically motivated thresholds
df['Health_Status'] = (
    (df['mean_hr'] > 220)      |   # extreme tachycardia
    (df['sdn'] < 0.05)        |   # critically low HRV
    (df['spo2_estimate'] < 95) |   # estimated hypoxia
).astype(int)                # 1 = Attention Needed, 0 = Normal

```

```

X = df[['mean_hr', 'sdnn', 'rmssd', 'pnn50', 'spo2_estimate']]
y = df['Health_Status']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

model = RandomForestClassifier()
model.fit(X_train, y_train)

print(classification_report(y_test, model.predict(X_test)))
joblib.dump(model, 'health_model.pkl')

```

Explanation: The label creation logic encodes clinical knowledge: a mean HR above 220 bpm is physiologically extreme; SDNN below 0.05 indicates dangerously low HRV; SpO2 below 95% indicates potential hypoxia. The 80/20 split and fixed random_state ensure reproducibility. The RandomForestClassifier is used with default hyperparameters for the initial phase.

5.2.5 *predict_from_java.py* – Python-Java Bridge

Purpose: Serves as the ML inference endpoint called by the Spring Boot API.

```

import sys, joblib, pandas as pd

# Load the trained model
model = joblib.load('health_model.pkl')

# Read HRV features from CLI arguments (passed by Spring Boot ProcessBuilder)
mean_hr, sdnn, rmssd, pnn50, spo2_estimate = [float(x) for x in sys.argv[1:6]]

# Construct input DataFrame
input_data = pd.DataFrame([
    {'mean_hr': mean_hr, 'sdnn': sdnn,
     'rmssd': rmssd, 'pnn50': pnn50,
     'spo2_estimate': spo2_estimate}
])

# Predict and output label to stdout (captured by Spring Boot)
prediction = model.predict(input_data)[0]
print('Attention Needed' if prediction == 1 else 'Normal')

```

Explanation: The Spring Boot PredictionService invokes this script via Java ProcessBuilder, passing the five HRV feature values as command-line arguments. The script loads the saved model, constructs a single-row DataFrame matching the training feature set, generates a prediction, and prints the label to stdout. Spring Boot reads this output and wraps it in a JSON response.

CHAPTER 6: TESTING

Testing of CardiAnalytics Pro was conducted at three levels: unit testing of individual Python modules, integration testing of the Python-PostgreSQL-Spring Boot chain, and system testing of the complete end-to-end pipeline.

6.1 Unit Testing

TC ID	Module	Test Case	Input	Expected Output	Status
UT-01	ECGFilter	Normal signal filtering	Synthetic sine wave at 60 Hz (noise)	Attenuated output below filter passband	Pass
UT-02	ECGFilter	Bandpass preserves QRS	MIT-BIH record 100, signal	Signal energy in 5–40 Hz band retained	Pass
UT-03	PanTompkins	R-peak count accuracy	MIT-BIH record 100	Detected peaks within 10% of annotated peaks	Pass
UT-04	PanTompkins	Refractory period	Synthetic double-peak signal	Only one peak detected per QRS	Pass
UT-05	HRVExtractor	SDNN computation	Known RR intervals [0.8, 0.82, 0.78, 0.81]	SDNN = $\text{std}([0.8, 0.82, 0.78, 0.81]) \approx 0.0158$	Pass
UT-06	HRVExtractor	pNN50 boundary	RR intervals all differing by exactly 0.05 s	pNN50 = 0 (not strictly > 0.05)	Pass
UT-07	db_writer	Successful DB insert	Valid feature dictionary	Row inserted in ecg_features table	Pass
UT-08	train_model	Label creation logic	mean_hr=250 (extreme)	Health_Status = 1 (Attention Needed)	Pass
UT-09	predict_from_java	Normal prediction output	mean_hr=75, sdnn=0.08, ...	stdout: 'Normal'	Pass
UT-10	predict_from_java	Attention prediction output	mean_hr=230, sdnn=0.03, ...	stdout: 'Attention Needed'	Pass

Table 8: Unit Test Cases

6.2 Integration Testing

TC ID	Test Scenario	Steps	Expected Result	Status
IT-01	Pipeline to DB	Run run_pipeline.py for record 100; query ecg_features table	Row with record_name='100' present in database	Pass
IT-02	Model to API	POST /api/predict with valid HRV JSON; JWT = Doctor role	JSON response with prediction label	Pass
IT-03	API Authentication	GET /api/features without JWT token	HTTP 401 Unauthorized	Pass
IT-04	Role Restriction	GET /api/features with Nurse JWT (restricted endpoint)	HTTP 403 Forbidden	Pass
IT-05	Power BI to DB	Refresh Power BI dashboard; check record count	All processed records visible in dashboard	Pass
IT-06	Pipeline exception handling	Provide invalid record name '999'	Error logged; pipeline continues to next record	Pass

Table 9: Integration Test Cases

6.3 System Testing

End-to-end system testing was conducted by running the complete pipeline from raw MIT-BIH data download through to Power BI dashboard visualisation. The system successfully processed records 100–109, populated the ecg_features table with 10 rows, trained the Random Forest model, and served predictions via the Spring Boot API. The Power BI dashboard correctly displayed HRV trends and prediction summaries for all processed records. All JWT-protected endpoints enforced role-based access as configured.

Note: The project is currently in progress. Full system testing on all 48 MIT-BIH records, integration of ml_predictions.csv and pipeline_results.csv into the dashboard, and testing of the LSTM multi-class classifier are planned for the next development phase.

CHAPTER 7: SUMMARY AND CONCLUSION

7.1 Summary

CardiAnalytics Pro is an end-to-end cardiac analytics platform built on the MIT-BIH Arrhythmia Database. The project successfully demonstrates that it is feasible to build a clinically deployable ECG analytics system within the scope of a B.Tech Major Project. The four-layer architecture – Python pipeline, PostgreSQL database, Spring Boot API, and Power BI dashboard – integrates signal processing, machine learning, data persistence, secure API services, and clinical visualisation into a coherent, modular system.

The Python pipeline implemented in this project covers the complete data lifecycle: from raw ECG signal acquisition (`download_dataset.py`, `signal_loader.py`), through noise removal (`ECGFilter`), R-peak detection (`PanTompkinsDetector`), and HRV feature extraction (`HRVFeatureExtractor`), to database persistence (`db_writer.py`) and CSV export (`run_pipeline.py`). The machine learning component (`train_model.py`) trains a Random Forest classifier and deploys it via a Python-Java bridge (`predict_from_java.py`) that integrates seamlessly with the Spring Boot REST API.

A comparative analysis with four contemporary peer-reviewed papers (CNN-LSTM-SE, ArrhythmiaNet, Lightweight CNN-LSTM, MS-CLDNet) revealed that while deep learning models achieve higher beat-level classification accuracy, none of them addresses clinical deployment, HRV analysis, data persistence, secure API services, or a clinical dashboard. CardiAnalytics Pro is unique in providing all these components in an integrated, open-source platform.

7.2 Conclusion

This project underscores a critical insight: the value of an ECG analytics system lies not only in its classification accuracy but in its clinical utility and deployability. A model that achieves 99% accuracy but is never deployed has zero clinical impact. CardiAnalytics Pro prioritises clinical integration, achieving approximately 89% binary classification accuracy with the Random Forest model while providing a complete, deployable, and secure platform.

The modular design of CardiAnalytics Pro ensures that the system can evolve. The Random Forest can be replaced with a more accurate LSTM or CNN-based multi-class classifier without altering the Spring Boot API or Power BI dashboard. Additional HRV features (frequency-domain, non-linear) can be added to the `feature_extractor` without restructuring the pipeline. The `ecg_features` database schema can be extended with additional columns without breaking existing queries.

CardiAnalytics Pro lays a strong foundation for a future clinical decision support system that could assist cardiologists in monitoring arrhythmia patterns, detecting high-risk events, and making data-driven treatment decisions.

CHAPTER 8: LIMITATIONS OF THE PROJECT AND FUTURE WORK

8.1 Current Limitations

- **Incomplete Dataset Coverage:** The current pipeline processes only records 100–109 (10 out of 48 MIT-BIH records). This limits the volume of training data available for the machine-learning model, which may affect generalisation.
- **Binary Classification Only:** The Random Forest model predicts only a binary health status (Normal vs. Attention Needed). It does not distinguish between specific arrhythmia classes such as PVC, PAC, AFib, or VF/VT.
- **Surrogate SpO2 Estimation:** The SpO2 estimate is computed using a linear formula ($98 - \text{sdnn} * 2$) that approximates blood oxygen levels based on HRV. This is not a physiologically validated SpO2 measurement and should be treated as a screening indicator only.
- **Single-Lead Processing:** The pipeline processes only the first ECG lead (channel 0) of each record. The second lead is currently ignored, which may miss lead-specific arrhythmia patterns.
- **Local Deployment Only:** The system is currently deployed on a local machine. It has not been deployed to a cloud environment or tested at scale with real patient data.
- **No LSTM / Deep Learning Integration:** The LSTM classifier planned as the primary arrhythmia classifier is currently in design phase. The existing model is the Random Forest baseline.
- **Static Dataset:** The system does not yet support real-time ECG streaming from IoT/wearable devices.

8.2 Future Work

- **Expand to All 48 MIT-BIH Records:** Process all 48 records to build a richer dataset and improve model training quality.
- **Integrate LSTM Multi-Class Classifier:** Replace the Random Forest with a trained LSTM or CNN-LSTM model capable of classifying 17+ arrhythmia classes, achieving > 98% accuracy comparable to contemporary deep learning approaches.
- **Real-Time ECG Streaming:** Extend the pipeline to process real-time ECG data streams from IoT devices (e.g., Raspberry Pi-based ECG sensors), enabling live arrhythmia detection.
- **Frequency-Domain HRV Analysis:** Add frequency-domain HRV metrics (LF power, HF power, LF/HF ratio) and non-linear measures (SD1, SD2, approximate entropy) to the feature extractor.
- **Cloud Deployment:** Deploy the Spring Boot API and PostgreSQL database on a cloud platform (AWS/GCP/Azure) with HTTPS, enabling remote access from hospital information systems.
- **SHAP Explainability:** Integrate SHAP (SHapley Additive exPlanations) to provide feature importance for each prediction, increasing clinician trust in the system.
- **Mobile Dashboard:** Develop a mobile-responsive Power BI dashboard or a React Native mobile app for bedside monitoring.
- **Formal Clinical Validation:** Conduct a clinical validation study with actual patient ECG data under IRB approval to assess real-world performance.

BIBLIOGRAPHY

- [1] A. L. Goldberger et al., "PhysioBank, PhysioToolkit, and PhysioNet: Components of a New Research Resource for Complex Physiologic Signals," *Circulation*, vol. 101, no. 23, pp. e215-e220, 2000.
- [2] J. Pan and W. J. Tompkins, "A Real-Time QRS Detection Algorithm," *IEEE Transactions on Biomedical Engineering*, vol. BME-32, no. 3, pp. 230-236, Mar. 1985.
- [3] Task Force of the European Society of Cardiology, "Heart Rate Variability: Standards of Measurement, Physiological Interpretation, and Clinical Use," *Circulation*, vol. 93, no. 5, pp. 1043-1065, 1996.
- [4] H. Li et al., "A CNN-LSTM-SE Model for Arrhythmia Classification Using ECG Signals," *IEEE Access*, 2024.
- [5] A. Ullah et al., "ArrhythmiaNet: A Novel Deep-Learning Framework for ECG Arrhythmia Classification," *Biomedical Signal Processing and Control*, 2022.
- [6] N. Alamatsaz et al., "A Lightweight Hybrid CNN-LSTM Explainable Model for ECG Classification," *IEEE Journal of Biomedical and Health Informatics*, 2022/2024.
- [7] Authors, "MS-CLDNet: Multi-Scale CNN-LSTM-Dense Network for ECG Arrhythmia Classification," 2025.
- [8] World Health Organization, "Cardiovascular Diseases (CVDs) Fact Sheet," WHO, 2023. [Online]. Available: [https://www.who.int/news-room/fact-sheets/detail/cardiovascular-diseases-\(cvds\)](https://www.who.int/news-room/fact-sheets/detail/cardiovascular-diseases-(cvds))
- [9] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825-2830, 2011.
- [10] Physionet MIT-BIH Arrhythmia Database v1.0.0. [Online]. Available: <https://physionet.org/content/mitdb/1.0.0/>
- [11] P. Warden and D. Situnayake, *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. O'Reilly Media, 2019.
- [12] I. Sommerville, *Software Engineering*, 10th ed. Pearson, 2016.
- [13] Spring Framework Documentation. [Online]. Available: <https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/>
- [14] PostgreSQL Global Development Group, "PostgreSQL Documentation," 2024. [Online]. Available: <https://www.postgresql.org/docs/>
- [15] Microsoft Corporation, "Power BI Documentation," 2024. [Online]. Available: <https://docs.microsoft.com/en-us/power-bi/>

APPENDIX

A. GitHub Repository Structure

```
CardiAnalytics/
├── 01_Python_Pipeline/
│   ├── download_dataset.py    # Downloads MIT-BIH records via wfdb
│   ├── signal_loader.py      # Loads ECG signal and annotations
│   ├── signal_filter.py      # ECGFilter: Butterworth bandpass
│   ├── rpeak_detector.py     # PanTompkinsDetector: R-peak detection
│   ├── feature_extractor.py  # HRVFeatureExtractor: SDNN/RMSSD/...
│   ├── db_writer.py          # save_to_db: PostgreSQL persistence
│   └── run_pipeline.py        # Master orchestrator
├── 03_SpringBoot_API/
│   └── cardi-analytics-api/
│       ├── src/main/java/cardi/analytics/api/
│       │   ├── controller/EcgFeatureController.java
│       │   ├── service/PredictionService.java
│       └── security/SecurityConfig.java
├── 04_PowerBI/
│   └── CardiAnalytics.pbix    # Power BI dashboard
├── 05_Dataset/
│   └── mitdb/                 # MIT-BIH raw ECG records
├── 07_Models/
│   ├── train_model.py        # Random Forest training
│   ├── predict_model.py      # Batch prediction → ml_predictions.csv
│   └── predict_from_java.py  # Python-Java bridge for Spring Boot
└── README.md
```

PROJECT & SDG MAPPING FORM

Student & Project Profile

- Student Full Name: Vishesh Gupta
- Student Enrollment No: 14214802722
- Department: Computer Science and Engineering
- Guide Name: Dr. Farzil Kidwai
- Project Title: CardiAnalytics Pro: End-to-End Cardiac Analytics Platform
- Project ID: ES-452

Primary SDG Alignment

- SDG 3: Good Health and Well-Being – CardiAnalytics Pro directly supports early detection of cardiovascular diseases through automated ECG analysis, arrhythmia classification, and real-time clinical alerting, contributing to improved cardiac health outcomes.
- SDG 9: Industry, Innovation and Infrastructure – The project demonstrates innovation in healthcare informatics by building a modular, deployable clinical analytics platform that bridges the gap between academic ML research and real-world hospital deployment.

Impact Mapping

1. Contribution Description

The system automates ECG signal processing, HRV feature extraction, arrhythmia classification, and clinical dashboard visualisation. By deploying a secure REST API with role-based access control, the system enables cardiologists to remotely query patient cardiac data – a direct contribution to SDG 3 (universal access to health services).

2. Key Indicators of Impact

10 MIT-BIH records processed; 5 HRV features extracted per record; binary health status classifier deployed; 7+ REST API endpoints secured; interactive Power BI dashboard with 4 panels implemented.

3. Personal Growth & Global Citizenship

Mapping my project to global goals made me realize that technology can directly contribute to improving healthcare and saving lives. It has motivated me to pursue a career where my technical skills create meaningful social impact alongside innovation.

Project Mapping Form: POs & PSOs

Student & Project Profile

- Student Full Name: Vishesh Gupta
- Student Enrollment No: 14214802722
- Department: Computer Science and Engineering
- Guide Name: Dr. Farzil Kidwai
- Project Title: CardiAnalyticsPro : End-to-End Cardi Analytics Platform
- Project ID: ES-452

Mapping:

Program Outcome	Level (1/2/3)	Specific task performed
PO 1: Engineering Knowledge	3 – High	Applied signal processing (Butterworth filter, Pan-Tompkins) and ML theory (Random Forest, HRV) to a real cardiac analytics system
PO 2: Problem Analysis	3 – High	Identified the clinical deployment gap in ECG research; defined 10 specific objectives addressing signal processing, ML, API, and visualisation
PO 3: Design/Development	3 – High	Designed and built a four-layer modular architecture: Python pipeline, PostgreSQL, Spring Boot API, Power BI dashboard
PO 4: Conduct Investigations	2 – Medium	Conducted comparative literature review of 4 contemporary papers; evaluated Random Forest performance with classification report
PO 5: Modern Tool Usage	3 – High	Used Python, SciPy, Scikit-learn, Spring Boot, PostgreSQL, Power BI, Postman, Git – industry-standard tools across multiple technology stacks
PO 6: Engineer and Society	2 – Medium	Project addresses global cardiac mortality burden; contributes to clinical decision support and SDG 3
PO 9: Individual and Team Work	2 – Medium	Individual project with advisor collaboration; modular code design supports future team extension
PO 10: Communication	3 – High	Comprehensive technical report with code documentation, comparative analysis tables, and system diagrams
PO 12: Life-long Learning	2 – Medium	Researched contemporary deep learning architectures (CNN-LSTM, SE networks) and clinical HRV standards
PSO 1: Emerging Technologies	3 – High	Applied Machine Learning, Signal Processing, REST API development, and BI visualisation to a healthcare problem

PSO 2: Innovative Solutions	3 – High	Developed a novel end-to-end clinical platform that uniquely integrates signal processing + ML + API + dashboard – not found in any surveyed paper
PSO 3: Research and Development	2 – Medium	Research paper comparison conducted; system positioned as a contribution to clinical informatics; future LSTM integration planned

Verification

- **Student Signature:** _____ **Date:** _____
- **Supervisor/Counsellor Signature:** _____ **Date:** _____



MAHARAJA AGRASEN INSTITUTE OF TECHNOLOGY

Department of Computer Science and Engineering

Major Project Guide Feedback Form

Student Details

Name(s) of Student(s): _____

Roll Number(s): _____

Project Title: _____

Guide Name: _____

Project Assessment Criteria

S. No.	Evaluation Parameter	Rating (1–10)	Comments
1	Problem Identification & Objective Clarity		
2	Literature Survey & Relevance		
3	Project Planning & Timeliness		
4	Design & Architecture		
5	Implementation & Technical Complexity		
6	Use of Tools/Technologies		
7	Testing & Validation		
8	Innovation & Originality		
9	Documentation Quality		
10	Presentation & Communication Skills		

Overall Comments / Suggestions

(Strengths, areas of improvement, technical depth, practical relevance, etc.)

Final

Recommendation

☐ Application Based

☐ Software

☐ Prototype Functional – Rating: Excellent, Good Satisfactory

☐ Prototype Incomplete – Rating: Excellent, Good Satisfactory

☐ Hardware

☐ Prototype Functional – Rating: Excellent, Good Satisfactory

☐ Prototype Incomplete – Rating: Excellent, Good Satisfactory

☐ Research Based

☐ Research Paper Published

☐ Research Paper Accepted

☐ Research Paper Communicated

☐ Patent Applied

☐ Patent Published

☐ Patent Granted

Guide Signature: _____

Date: _____